App Context Extraction: Roulette Tactician

Target Platform: Android (Samsung Galaxy S26 Ultra) **Orientation:** Landscape Mode (Exclusive/Primary) **Tech Stack:** Kotlin, Jetpack Compose, Room Database, Hilt, Coroutines

1. Executive Summary

"Roulette Tactician" is a strategic companion app for Roulette players. It does not simulate the game (RNG) but rather tracks real-world spins, calculates "Hot" and "Cold" numbers based on a specific algorithm (Frequency + Recency), and generates a heatmap on a betting table. Users drag and drop chips onto this table to plan bets.

2. Device Specification (S26 Ultra Context)

- **Form Factor:** ~6.8+ inch AMOLED.
- **Resolution:** QHD+ (High Density).
- **Aspect Ratio:** Wide (approx 19.3:9 or 20:9).
- **Input:** Touch (Multi-touch for chips) & S-Pen (Hover tooltips).
- **Layout Strategy:**
  - **Left Pane (20%):** Spin History (Vertical List) & Stats.
  - **Center Pane (65%):** The Betting Table (Canvas). Maximize this space.
  - **Right Pane (15%):** Chip Rack (Vertical) & Action Buttons (Spin, Clear, Settings).

3. Core Algorithms (Crucial for Porting)

A. The Heatmap Logic (The "Brain")

The app colors numbers on the board based on a ranking system.

- **Input:** `spinHistory` (List<Int>), `hotNumbers` (Manual Overrides), `coldNumbers` (Manual Overrides), `window` (Int, e.g., last 50 spins).
- **Logic:**
  1. **Manual Check:** If a number is in `hotNumbers`, Color = Bright Yellow (`#facc15`). If in `coldNumbers`, Color = Deep Navy (`#172554`).
  2. **Ranking:** For all other numbers:
     - Calculate `Frequency` (Count in window).

- Calculate `Recency` (Index of last occurrence normalized 0.0-1.0).
- `Score = Frequency + (Recency * 0.99)`.
- Sort numbers by Score Descending.

3. **Gradient Generation:**
   - **Top 50% (Hot):** Interpolate color from **Slate Gray** to **Golden Orange**.
   - **Bottom 50% (Cold):** Interpolate color from **Slate Gray** to **Royal Blue**.
   - *Note:* Cold intensity scales with total spin count (prevents bright blue board at start).

B. The Geometric Snap System (The Board)

The board is **not** a grid of buttons. It is a coordinate system.

- **Grid Structure:** 3 Rows (Top, Mid, Bot) x 12 Columns.
- **Touch Handling:** When a user drags a chip to `(x,y)`:
  - Convert screen coordinates to "Board Coordinates".
  - **Hit Testing:** Check distance to specific "Snap Points":
    - **Straight:** Center of a number cell.
    - **Split:** Edge between two numbers (Vertical or Horizontal).
    - **Corner:** Intersection of 4 numbers.
    - **Street:** Top edge of a row.
    - **Line:** Intersection of 2 rows at the top edge.
    - **Outside Bets:** Defined rectangles for Even/Odd, Red/Black, Dozens.
- **Android Implementation:** Use `Canvas` within `BoxWithConstraints`. Use `detectDragGestures`.

4. Data Models (Kotlin)

```
// Represents a recorded spin
@Entity(tableName = "spins")
data class SpinResult(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val sessionId: Long,
    val number: Int, // 0-36
    val timestamp: Long
)

// Represents a chip placed on the board
data class PlacedChip(
```

```
    val id: Long = System.currentTimeMillis(),
    val value: Int, // e.g., 1, 5, 25
    val xPercent: Float, // Relative X position (0.0 - 1.0) on Board Can
    val yPercent: Float, // Relative Y position (0.0 - 1.0) on Board Can
    val betId: String // e.g., "num-32", "split-17-20", "doz-1"
)

// Session Data for Save/Load
data class StrategySession(
    val name: String,
    val hotOverrides: List<Int?>,
    val coldOverrides: List<Int?>,
    val analysisWindow: Int
)
```

5. UI/UX Requirements

A. The Board (Center)

- Must support **Pinch-to-Zoom** and **Pan** (use `Modifier.transformable`).
- Rendering:
  - Draw the grid lines (White, low opacity).
  - Draw Number backgrounds (Gradient calculated from Heatmap logic).
  - Draw Numbers (Text).
  - Draw Chips (Z-Index highest). Chips should cast a shadow.

B. Controls (Sidebars)

- **Keypad:** A modal or dedicated bottom sheet for entering spin results (0-36).
- **Chip Selector:** Vertical scroll list of chips (1, 5, 25, 100, 500, 1k).
- **Stats Panel:**
  - Display "Last 20" spins horizontally or vertically.
  - Show current Total Bet value.

C. S-Pen Integration (S26 Ultra Specific)

- **Hover Events:** Using `Modifier.pointerInput`, detect `PointerEventType.Enter/Exit`.
- **Action:** When S-Pen hovers over a number:
  - Show tooltip: "Number 17: Hit 3 times in last 50 spins (Rank #4)".

6. Migration Checklist for AI Agent

1. **Setup:** Create Android project with Compose & Hilt.
2. **Database:** Setup Room for `SpinHistory`.
3. **Logic Port:** Convert the JS `useMemo` stats logic to a Kotlin `ViewModel` function `calculateHeatmap(spins: List<Int>)`.
4. **Canvas Engine:** Build the `RouletteBoard` composable.
   - *Crucial:* Port the `findSnapTarget` function from the JS code. It uses normalized coordinates (0.0 - 1.0) relative to grid dimensions. Keep this normalized approach to handle different screen sizes.
5. **State:** Use `MutableStateFlow<List<PlacedChip>>` for the chips on board.